

JavaScript Introduction

Theory Assignment

Question 1: What is JavaScript? Explain the role of JavaScript in web development.

JavaScript (often abbreviated as JS) is a high-level, versatile programming language primarily used to create interactive and dynamic content on websites. It is the only programming language native to web browsers, making it a fundamental technology of the World Wide Web alongside HTML and CSS.

While HTML structures the web page and CSS styles it, JavaScript brings it to life by allowing users to interact with the page elements.

The "Big Three" of Web Development

To understand JavaScript's role, it is helpful to visualize it as part of the standard web technology stack:

1. HTML (HyperText Markup Language): The skeleton. It defines the structure and content of the page (paragraphs, headings, data tables, or embedded images).
2. CSS (Cascading Style Sheets): The skin and clothing. It defines the presentation (colors, fonts, layout, and responsiveness).
3. JavaScript: The muscles and brain. It defines the behavior (what happens when a user clicks a button, submits a form, or scrolls).

The Role of JavaScript in Web Development

JavaScript has evolved significantly from a simple scripting language to a powerful tool capable of running complex applications. Its role can be categorized into two main areas:

1. Client-Side Development (Frontend)

This is the code that executes in the user's web browser. JavaScript is responsible for:

- Interactivity: Handling events like clicks, mouse movements, and key presses.
- DOM Manipulation: Changing the HTML structure or CSS styles on the fly without reloading the page. For example, clicking "Dark Mode" changes the background color instantly.
- Data Validation: Checking if a user has filled out a form correctly before sending it to the server (e.g., ensuring an email address contains an "@" symbol).

- Asynchronous Communication (AJAX/Fetch): Fetching data from a server in the background to update parts of a page without a full reload. This is how feeds on social media sites update automatically.
- Web Applications: Powering Single Page Applications (SPAs) like Gmail, Google Maps, or Trello, which feel more like desktop software than traditional websites.

2. Server-Side Development (Backend)

With the advent of Node.js, JavaScript can now run outside of the browser on a server. This allows developers to use a single language for the entire stack (Full Stack Development). On the server, JavaScript is used for:

- API Development: Creating the endpoints that frontend applications talk to.
- Database Operations: Reading and writing data to databases (like MongoDB).
- Server Logic: Handling authentication, file processing, and business logic.

Key Features of JavaScript

- Interpreted Language: The code is executed line-by-line by the browser, meaning it doesn't need to be compiled beforehand.
- Dynamically Typed: Variables do not require a specific data type definition; the type is determined automatically during execution.
- Event-Driven: It is designed to respond to user actions (events) rather than just running a linear script from top to bottom.
- Huge Ecosystem: It has the largest package registry in the world (NPM) and massive community support with frameworks like React, Vue, and Angular.

Feature	HTML	CSS	JavaScript
Analogy	Skeleton/Structure	Skin/Style	Muscles/Behavior
Function	Holds content	Looks good	Does things
Static/Dynamic	Static	Static	Dynamic

Example	<code><button>Click</button></code>	<code>button { color: red; }</code>	<code>btn.onclick = alert('Hi!');</code>
---------	---	-------------------------------------	--

Question 2: How is JavaScript different from other programming languages like Python or Java?

JavaScript, Python, and Java are three of the most popular programming languages in the world, but they were built for very different purposes and operate in distinct ways.

Here is a breakdown of how JavaScript stands apart from its peers.

1. Execution Environment: The "Where"

The most defining difference is where these languages naturally live.

- **JavaScript:** It is the only language native to the web browser. If you want code to run on a user's computer inside Chrome or Safari to make a webpage interactive, it *must* be JavaScript. While Node.js allows it to run on servers now, its roots and primary monopoly are in the frontend.
- **Python:** A general-purpose scripting language that usually runs directly on an operating system (Linux, Windows, macOS). It requires an interpreter installed on the machine.
- **Java:** Runs on the Java Virtual Machine (JVM). This allows Java code to be "Write Once, Run Anywhere"—meaning the same code can run on any device that has a JVM, from a massive server to an Android phone or a washing machine.

2. Concurrency Model: Handling Multiple Tasks

This is often the most confusing technical difference for beginners.

- **JavaScript (Event Loop):** JavaScript is single-threaded. It can only do one thing at a time. However, it uses an Event Loop to handle long tasks (like fetching data) in the background without freezing the interface. It says, "Go get this data, and call me back when you're done," and immediately moves to the next line of code.

- Java (Multi-threading): Java is multi-threaded. If it has two heavy tasks, it can spawn two separate "threads" (workers) to handle them simultaneously. This makes it powerful for CPU-intensive tasks but more complex to manage.
- Python: generally uses a single thread but can simulate concurrency. However, it is often slower than Java at heavy parallel processing due to the Global Interpreter Lock (GIL).

3. The Typing System: Strictness

This dictates how the language handles data types (like numbers vs. text).

- JavaScript (Dynamic & Weak): You don't have to define what a variable is. You can create a variable holding a number and then suddenly switch it to hold text.
 - *Example:* `let x = 5; x = "Hello";` (This works fine).
- Python (Dynamic & Strong): Like JS, you don't define types upfront. However, it is "stronger" regarding type mixing. If you try to add "5" (text) + 5 (number), Python will throw an error, whereas JavaScript might try to combine them into "55".
- Java (Statically Typed): Very strict. You must define exactly what every variable is before you use it. If you say `int x = 5;`, `x` can never be anything but an integer. This makes code more verbose but significantly reduces bugs in large systems.

4. Object-Oriented Style: Classes vs. Prototypes

While all three support Object-Oriented Programming (OOP), JavaScript does it differently.

- Java & Python (Class-based): You create a "Blueprint" (Class) and build objects from it. It is a rigid, top-down structure.
- JavaScript (Prototype-based): There are no true classes (though modern JS adds "class" keywords to look like them). Instead, objects inherit directly from other objects (prototypes). It is more flexible but can be confusing if you are coming from a Java background.

Feature	JavaScript	Python	Java

Primary Domain	Web Frontend (Browser)	Data Science, AI, Scripting	Enterprise Backends, Android
Typing	Dynamic & Weak	Dynamic & Strong	Static & Strong
Speed	Fast (V8 Engine)	Slower (Interpreted)	Very Fast (Compiled to Bytecode)
Concurrency	Event Loop (Async)	Threading (with GIL limits)	Multi-threading (Native)
Syntax Style	C-style (curly braces <code>{}</code>)	Indentation (whitespace)	C-style (curly braces <code>{}</code>)

Question 3: Discuss the use of `<script>` tag in HTML. How can you link an external JavaScript file to an HTML document?

The `<script>` tag is one of the most fundamental elements in HTML, serving as the primary way to insert or reference JavaScript code within an HTML document. It's what makes a static webpage dynamic and interactive.

1. Uses of the `<script>` Tag

The `<script>` tag has two main ways it's used to include JavaScript:

A. Inline/Internal JavaScript

You can write the JavaScript code directly between the opening and closing tags.

- When to use: This is usually reserved for small, simple scripts that are only relevant to a single HTML page.

Example:

```
<script>

function showAlert() {

    alert("Hello from internal JavaScript!");

}

</script>

<button onclick="showAlert()">Click Me</button>
```

2. Linking an External JavaScript File

To link an external JavaScript file, you use the `<script>` tag with the `src` (source) attribute.

The `src` attribute specifies the path to your JavaScript file.

The Syntax

Assuming your JavaScript file is named `main.js` and is located in the same directory as your HTML file, the syntax is:

```
<script src="main.js"></script>
```

Lab Assignment(Task)

Create a simple HTML page and add a `<script>` tag within the page. Write JavaScript code to display an alert box with the message “Welcome to Javascript!” when the page is loaded.

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>JavaScript Alert</title>

  <script>

    // 2. The JavaScript code is written directly inside the <script> tags.

    // The alert() function is executed immediately when the script is loaded

    // by the browser, which happens as soon as the <head> section is processed.

    alert("Welcome to Javascript!");

  </script>

</head>

<body>

  <h1>Page Content</h1>

  <p>This content will appear after you dismiss the "Welcome to Javascript!" alert
  box.</p>

</body>

</html>
```

Variables and Data Types

Theory Assignment

Variables and Data Types

Question 1: What are variables in JavaScript? How do you declare a variable using var, let, and const?

Variables are fundamental building blocks in JavaScript (and almost every programming language). They serve as named containers for storing data values.

Think of a variable as a labeled box in your computer's memory. The label is the variable name (e.g., `age`), and the content inside the box is the value (e.g., `25`).

1. Defining Variables in JavaScript

JavaScript is dynamically typed, meaning you don't have to specify what type of data (like a number or a string) the variable will hold when you declare it. It figures out the type automatically during execution.

In modern JavaScript, there are three primary keywords used to declare variables: `var`, `let`, and `const`. The choice between these keywords is critical as it determines the variable's scope (where it can be accessed) and whether its value can be changed.

2. The Three Declaration Keywords

A. `var` (The Legacy Keyword) is the oldest keyword for variable declaration, and while it still works, it has been largely superseded by `let` and `const` due to its confusing scoping rules.

Feature	Description
Scope	Function-Scoped (or Globally-Scoped). A variable declared with <code>var</code> inside a code block (like an <code>if</code> statement or <code>for</code> loop) is accessible <i>outside</i> that block, which can lead to bugs.

Reassignment	Yes. The value can be changed later.
Redeclaration	Yes. You can declare the same variable multiple times in the same scope without an error.
Hoisting	Yes, with initialization. The variable is hoisted (moved to the top of its scope) and initialized with the value <code>undefined</code> .

```
var greeting = "Hello";
```

```
var greeting = "Hi"; // Redeclaration is allowed (bad practice)
```

```
if (true) {
```

```
    var message = "Function Scoped";
```

```
}
```

```
console.log(message); // Outputs: "Function Scoped" (var "leaks" outside the block)
```

```
var greeting = "Hello";
```

```
var greeting = "Hi"; // Redeclaration is allowed (bad practice)
```

```
if (true) {
```

```
    var message = "Function Scoped"; }
```

```
console.log(message); // Outputs: "Function Scoped" (var "leaks" outside the block)
```

B. `let` (The Modern, Changeable Variable)

`let` was introduced in ECMAScript 2015 (ES6) to solve the scoping problems of `var`. It is the preferred way to declare variables whose value you expect to change.

Feature	Description

Scope	Block-Scoped. A variable declared with <code>let</code> inside a pair of curly braces <code>{ }</code> (a block) is only accessible within that block.
Reassignment	Yes. The value can be changed.
Redeclaration	No. Trying to declare the same variable twice will cause a Syntax Error.
Hoisting	Yes, without initialization. It is hoisted, but accessing it before its declaration results in a Reference Error (this area is called the Temporal Dead Zone).

```
let count = 10;
```

```
count = 20; // Reassignment is allowed
```

```
// let count = 30; // ERROR: SyntaxError: 'count' has already been declared
```

```
if (true) {
```

```
    let blockVariable = "Block Scoped";    }
```

```
// console.log(blockVariable); // ERROR: ReferenceError: blockVariable is not defined
```

C. `const` (The Constant, Unchangeable Variable)

`const` was also introduced in ES6 and stands for "constant." It is used for variables whose values should never change after they are initialized.

Feature	Description
Scope	Block-Scoped (same as <code>let</code>).

Reassignment	No. You must assign a value when you declare it, and you cannot change that value later.
Redeclaration	No.
Hoisting	Yes, without initialization (Temporal Dead Zone).

```
const PI = 3.14159;
```

```
// PI = 3.14; // ERROR: TypeError: Assignment to constant variable
```

```
const user = { name: "Alice" };
```

```
// user = {}; // ERROR: Cannot reassign the 'user' variable itself
```

// IMPORTANT NOTE: You CAN change the PROPERTIES of an object declared with const:

```
user.name = "Bob"; // This is ALLOWED because the 'user' variable still points to the same object in memory.
```

```
console.log(user.name); // Outputs: "Bob"
```

3. Best Practices (When to Use Which)

In modern JavaScript development, a standard approach is to prioritize the following:

1. Use **const** by default: If you don't plan on changing the variable's value, use **const**. This improves code safety and makes your intentions clearer.
2. Use **let** when necessary: Only use **let** if you know the variable's value will change later (e.g., loop counters, running totals, or variables that need to be updated).
3. Avoid **var**: You should almost never use **var** in new code due to its less intuitive function scoping and redeclaration capabilities.

Question 2: Explain the different data types in JavaScript. Provide examples for each.

JavaScript features a rich set of data types used to store and manipulate different kinds of information. These data types are categorized into two main groups: Primitive Types and Non-Primitive (or Reference) Types.

1. Primitive Data Types

Primitive types are single, simple data values. They are immutable (cannot be changed) and are stored directly in the variable location. There are seven primitive data types in JavaScript:

A. Number

Represents both integer values and floating-point numbers. There is no distinction between integers and floats as there is in languages like Java or C++.

Example	Description
<code>let integer = 10;</code>	An integer value.
<code>let floatNum = 3.14;</code>	A floating-point value.
<code>let bigNum = 1.2e+6;</code>	Scientific notation (1,200,000).
<code>let notANum = NaN;</code>	A special value representing "Not a Number" (e.g., from an impossible calculation like <code>0 / 0</code>).

B. String

Represents a sequence of characters, used for text. Strings can be defined using single quotes (`' ... '`), double quotes (`" ... "`), or backticks (`` ... ``).

Example	Description
<code>let name = "Alice";</code>	Defined with double quotes.
<code>let phrase = 'Hello World';</code>	Defined with single quotes.
<code>let greeting = `Hello, \${name}`;</code>	Template literals (using backticks) allow for embedded variables (interpolation).

C. Boolean

Represents a logical entity and can only have one of two values: `true` or `false`. They are essential for conditional logic and control flow

Example	Description
<code>let isAdult = true;</code>	Represents truth.
<code>let hasPermission = false;</code>	Represents falsehood.
<code>let isEqual = (5 === 5);</code>	The result of a comparison operator is a boolean (<code>true</code>).

D. Undefined

Represents a variable that has been declared but has not yet been assigned a value.

Example	Description
<code>let myVariable;</code>	Declared but not initialized. Its value is automatically undefined .
<code>typeof myVariable;</code>	Returns "undefined" .

E. Null

Represents the intentional absence of any object value or meaningful data. It must be explicitly assigned.

Example	Description
<code>let data = null;</code>	The variable is intentionally empty.
<code>typeof null;</code>	NOTE: This returns "object" , which is a historical bug in JavaScript, not a true reflection of its type.

F. Symbol (ES6)

Represents a unique identifier. Symbols are primarily used to create unique object property keys that avoid name collisions.

Example	Description
---------	-------------

<code>const id = Symbol('user_id');</code>	Creates a unique symbol value.
<code>let obj = {}; obj[id] = 1;</code>	Uses the symbol as a unique property key.

2. Non-Primitive (Reference) Data Types

Non-primitive types are complex data structures that can hold multiple values and are stored by reference. This means the variable doesn't hold the object itself, but a pointer (reference) to where the object is stored in memory.

A. Object

The fundamental non-primitive type. Everything in JavaScript that is not a primitive type is essentially an object. Objects are used to store key-value pairs (properties and methods).

Example	Description
<code>let person = { name: "John", age: 30 };</code>	A standard object literal.

B. Array

A special type of object used to store ordered lists of values (which can be of any data type).

Example	Description
<code>let colors = ["red", "green", "blue"];</code>	An array of strings.

C. Function

Although functions are callable code blocks, in JavaScript, they are technically a special type of object (`typeof function` returns `"function"`). They can be stored in variables and passed around like any other value.

Example	Description
<code>function sayHi() { return "Hi"; }</code>	A function declaration.
<code>let greeting = sayHi;</code>	The function itself is treated as a value.

The `typeof` Operator

You can determine the data type of a variable using the built-in `typeof` operator:

```
console.log(typeof 42);      // Output: "number"
console.log(typeof "text");  // Output: "string"
console.log(typeof true);    // Output: "boolean"
console.log(typeof undefined); // Output: "undefined"
console.log(typeof null);    // Output: "object" (The historical error)
console.log(typeof {a: 1});  // Output: "object"
console.log(typeof [1, 2]);  // Output: "object" (Arrays are objects)
console.log(typeof function(){}); // Output: "function"
```

Question 3: What is the difference between undefined and null in JavaScript?

The difference between `undefined` and `null` is a common source of confusion in JavaScript, as both represent an "empty" or "non-existent" value, but they convey different meanings and arise in different contexts.

Feature	undefined	null
Definition	Value not yet assigned.	Intentional absence of value.
Assignment	Automatic (by the engine).	Manual (by the developer).
Type (Actual)	Primitive (undefined).	Primitive (null).
typeof Result	"undefined"	"object"
== Comparison	undefined == null is true.	null == undefined is true.

Lab Assignment (Task)

Write a javascript program to declare variables for different data types (string,number,boolean,null and undefined). Log the values of the variables and their types to the console using console.log().

// 1. String Data Type

// Used for sequences of characters (text)

```
let userName = "Alex Johnson";
```

// 2. Number Data Type

// Used for both integers and floating-point values

```
let productPrice = 49.99;
```

// 3. Boolean Data Type

// Used for true/false values, essential for logic and conditions

```
const isAvailable = true;
```

// 4. Null Data Type

// Used to intentionally represent the absence of an object value (explicitly set)

```
let shoppingCart = null;
```

// 5. Undefined Data Type

// Used for variables that have been declared but not assigned a value (automatically set)

```
let favoriteColor; // Declared but not initialized
```

// 6. Object Data Type (Reference Type)

// Used for storing collections of key-value pairs

```
const userProfile = {
```

```
    age: 28,
```

```
    city: "New York"
```

```
};
```

// 7. Array Data Type (a specialized Object)

// Used for storing ordered lists of values

```
const fruits = ["Apple", "Banana", "Cherry"];
```

// --- Logging Values and Types to the Console ---

```
console.log("--- Data Type Demonstration ---");
```

```
console.log("-----");
```

// Log String

```
console.log(`Variable: userName\nValue: ${userName}\nType: ${typeof userName}\n`);
```

// Log Number

```
console.log(`Variable: productPrice\nValue: ${productPrice}\nType: ${typeof productPrice}\n`);
```

```
// Log Boolean
```

```
console.log(`Variable: isAvailable\nValue: ${isAvailable}\nType: ${typeof isAvailable}\n`);
```

```
// Log Null (Note: typeof null returns "object" due to a historical bug)
```

```
console.log(`Variable: shoppingCart\nValue: ${shoppingCart}\nType: ${typeof shoppingCart}\n`);
```

```
// Log Undefined
```

```
console.log(`Variable: favoriteColor\nValue: ${favoriteColor}\nType: ${typeof favoriteColor}\n`);
```

```
// Log Object
```

```
console.log(`Variable: userProfile\nValue:`, userProfile, `\nType: ${typeof userProfile}\n`);
```

```
// Log Array
```

```
console.log(`Variable: fruits\nValue:`, fruits, `\nType: ${typeof fruits}\n`);
```

JavaScript Operators

Theory Assignment

Question 1: What are the different types of operators in JavaScript? Explain with examples.

- **Arithmetic operators**
- **Assignment operators**
- **Comparison operators**
- **Logical operators**

JavaScript operators are special symbols used to perform operations on values and variables. You have asked for an explanation of four fundamental categories, which are essential for any JavaScript program.

1. Arithmetic Operators

Operator	Name	Description	Example	Result
+	Addition	Adds two operands.	let x = 5 + 3;	x is 8
-	Subtraction	Subtracts the right operand from the left.	let y = 10 - 4;	y is 6
*	Multiplication	Multiplies two operands.	let z = 6 * 2;	z is 12
/	Division	Divides the left operand by the right.	let a = 15 / 3;	a is 5

%	Modulus	Returns the remainder of a division.	let b = 10 % 3;	b is 1
++	Increment	Increases an integer value by 1.	let c = 5; c++;	c is 6
--	Decrement	Decreases an integer value by 1.	let d = 5; d--;	d is 4
**	Exponentiation	Raises the first operand to the power of the second. (ES6+)	let e = 2 ** 3;	e is 8

2. Assignment Operators

These are used to assign values to variables. The simple assignment operator is `=`, but there are compound assignment operators that perform an arithmetic operation and an assignment simultaneously.

Operator	Name	Shorthand for	Example	Result
=	Assignment	N/A	let x = 10;	x is 10
+=	Addition Assignment	x = x + y	let x = 5; x += 3;	x is 8
-=	Subtraction Assignment	x = x - y	let x = 10; x -= 4;	x is 6

<code>*=</code>	Multiplication Assignment	<code>x = x * y</code>	let x = 6; x *= 2;	x is 12
<code>/=</code>	Division Assignment	<code>x = x / y</code>	let x = 15; x /= 3;	x is 5
<code>%=</code>	Modulus Assignment	<code>x = x % y</code>	let x = 10; x %= 3;	x is 1

3. Comparison Operators

These are used to compare two values, and they always return a boolean result: either **true** or **false**.

Operator	Name	Description	Example	Result
<code>==</code>	Equal To (Loose)	Checks if values are equal, ignoring the data type (Type Coercion occurs).	<code>5 == "5"</code>	true
<code>===</code>	Strict Equal To	Checks if values are equal AND if they are of the same data type. (Recommended)	<code>5 === "5"</code>	false
<code>!=</code>	Not Equal To (Loose)	Checks if values are not equal, ignoring the data type.	<code>5 != 8</code>	true
<code>!==</code>	Strict Not Equal To	Checks if values or types are different. (Recommended)	<code>5 !== "5"</code>	true

>	Greater Than	Checks if the left operand is greater than the right.	10 > 5	true
<	Less Than	Checks if the left operand is less than the right.	10 < 5	false
>=	Greater Than or Equal To	Checks if the left operand is greater than or equal to the right.	5 >= 5	true
<=	Less Than or Equal To	Checks if the left operand is less than or equal to the right.	5 <= 4	false

4. Logical Operators

These are used to determine the logic between variables or values, often combining two or more boolean expressions.

Operator	Name	Description	Example	Result
&&	AND	Returns true if both operands are true .	(5 > 3) && (2 < 4)	true
,		,	OR	Returns true if at least one operand is true .

!	NOT	Reverses the boolean result of the operand.	!(5 === 5)	false
---	-----	---	------------	-------

Question 2: What is the difference between == and === in JavaScript?

Loose Equality Operator (==)

The double equals sign (==) compares values but performs type coercion (automatic type conversion) before making the comparison.

Key Behavior:

- Checks Value Only: It attempts to convert the operands to a common data type *before* comparison.
- Type Coercion: If the operands are of different types (e.g., a **string** and a **number**), JavaScript will try to convert one or both to make them compatible.
- Result: It can often lead to unexpected **true** results, which makes code less predictable and harder to debug.

Expression	Explanation	Result
5 == 5	Values are equal, types are equal (number).	true
5 == "5"	JavaScript converts the string "5" to the number 5 before comparing.	true
0 == false	JavaScript converts false to 0 and compares 0 == 0.	true

<code>null == undefined</code>	JavaScript's rules define these two as loosely equal.	true
--------------------------------	---	------

Strict Equality Operator (`===`)

The triple equals sign (`===`) compares values without performing any type coercion.

Key Behavior:

- Checks Value AND Type: It returns `true` only if both the value and the data type of the two operands are identical.
- No Coercion: If the operands are of different types, the comparison immediately returns `false`.
- Result: It is faster, more predictable, and is the recommended way to compare equality in modern JavaScript development.

Expression	Explanation	Result
<code>5 === 5</code>	Values are equal, types are equal (number).	true
<code>5 === "5"</code>	Values are equal, but the types (<code>number</code> vs. <code>string</code>) are different.	false
<code>0 === false</code>	Values are different, and types (<code>number</code> vs. <code>boolean</code>) are different.	false
<code>null === undefined</code>	Values are different, and types are different (<code>object</code> vs. <code>undefined</code>).	false

Diff Between == & ===

Feature	== (Loose Equality)	=== (Strict Equality)
Type Check	No (Ignores type)	Yes (Requires types to match)
Coercion	Yes (Converts types automatically)	No (No conversion)
Best Practice	Avoid (can be unpredictable)	Recommended (predictable and safer)

Lab Assignment(Task)

Create a JavaScript program to perform the following:

Add, subtract, multiply, and divide two numbers using arithmetic operators.

Use comparison operators to check if two numbers are equal and if one number is greater than the other.

Use logical operators to check if both conditions (e.g., $a > 10$ and $b < 5$) are true.

```
// --- 1. Variable Initialization ---
```

```
// Define two numbers for the operations
```

```
const num1 = 20;
```

```
const num2 = 5;
```

```
console.log(`--- Initial Numbers ---`);
```

```
console.log(`First Number (num1): ${num1}`);

console.log(`Second Number (num2): ${num2}\n`);

// --- 2. Arithmetic Operators ---

console.log("--- Arithmetic Operations ---");

// Addition (+)

let sum = num1 + num2;

console.log(`Addition (${num1} + ${num2}): ${sum}`);

// Subtraction (-)

let difference = num1 - num2;

console.log(`Subtraction (${num1} - ${num2}): ${difference}`);

// Multiplication (*)

let product = num1 * num2;

console.log(`Multiplication (${num1} * ${num2}): ${product}`);

// Division (/)

let quotient = num1 / num2;

console.log(`Division (${num1} / ${num2}): ${quotient}`);

// Modulus (%) - Finding the remainder

let remainder = num1 % num2;

console.log(`Modulus (${num1} % ${num2}): ${remainder}\n`);

// --- 3. Comparison Operators ---

console.log("--- Comparison Operations ---");

// Strict Equality (===) - Check if two numbers are equal

let isEqual = (num1 === num2);
```

```
console.log(`Is ${num1} strictly equal to ${num2} (num1 === num2)? ${isEqual}`); // false
```

```
// Greater Than (>) - Check if one number is greater than the other
```

```
let isGreater = (num1 > num2);
```

```
console.log(`Is ${num1} greater than ${num2} (num1 > num2)? ${isGreater}`); // true
```

```
// Less Than (<) - For completeness
```

```
let isLess = (num1 < num2);
```

```
console.log(`Is ${num1} less than ${num2} (num1 < num2)? ${isLess}\n`); // false
```

```
// --- 4. Logical Operators ---
```

```
console.log("--- Logical Operations ---");
```

```
// Define two custom conditions for the logical checks
```

```
const conditionA = (num1 > 10); // 20 > 10, which is TRUE
```

```
const conditionB = (num2 < 5); // 5 < 5, which is FALSE
```

```
console.log(`Condition A (num1 > 10): ${conditionA}`);
```

```
console.log(`Condition B (num2 < 5): ${conditionB}`);
```

```
// Logical AND (&&) - Checks if BOTH conditions are TRUE
```

```
let resultAND = conditionA && conditionB;
```

```
console.log(`Result of AND (A && B): ${resultAND}`); // true && false => FALSE
```

```
// Logical OR (||) - Checks if AT LEAST ONE condition is TRUE
```

```
let resultOR = conditionA || conditionB;
```

```
console.log(`Result of OR (A || B): ${resultOR}`); // true || false => TRUE
```

```
// Logical NOT (!) - Reverses the boolean value
```

```
let resultNOT = !conditionA;
```

```
console.log(`Result of NOT (!A): ${resultNOT}`); // !true => FALSE
```

Control Flow (If-Else, Switch)

Theory Assignment

Question 1: What is control flow in JavaScript? Explain how if-else statements work with an example.

Control flow is the order in which individual statements, instructions, or function calls are executed or evaluated in a program. In a simple script, code execution starts from the first line and proceeds sequentially to the last line.

However, control flow statements allow you to break this simple linear path by enabling the program to make decisions (like `if-else`) and repeat actions (like loops).

Types of Control Flow Statements

1. Conditional Statements: Execute different blocks of code based on certain conditions (`if`, `else`, `else if`, `switch`).
2. Loop Statements: Execute a block of code repeatedly until a condition is met (`for`, `while`, `do-while`).

The `if-else` Statement

The `if-else` statement is the most fundamental type of conditional control flow. It allows your program to test a condition and execute different code blocks depending on whether that condition evaluates to `true` or `false`.

How it Works:

1. The `if` block: The program first evaluates the condition inside the parentheses `()`. If the condition is `true`, the code block inside the curly braces `{}` immediately following the `if` statement is executed.
2. The `else if` block (Optional): If the initial `if` condition is `false`, the program moves to check any optional `else if` conditions in order. It executes the first `else if` block whose condition is `true`. You can have multiple `else if` blocks.
3. The `else` block (Optional): If all preceding `if` and `else if` conditions are `false`, the code inside the final `else` block is executed. This block acts as a default action.

```
// Variable declaration

const userAge = 18;

const hasSubscription = false;

if (userAge >= 18 && hasSubscription === true) {

    // Condition 1: Must be 18 or older AND have a subscription

    console.log("Welcome! You have full access to premium content.");

} else if (userAge >= 18 && hasSubscription === false) {

    // Condition 2: Must be 18 or older BUT does NOT have a subscription

    console.log("You have standard access. Please subscribe for premium features.");

} else if (userAge >= 13 && userAge < 18) {

    // Condition 3: Must be 13 or older but younger than 18

    console.log("Access limited to parental controls. Adult supervision required.");

} else {

    // Default action: If none of the above conditions are met (age < 13)

    console.log("Access denied. Minimum age requirement not met.");    }

}
```

Output: You have standard access. Please subscribe for premium features.

Question 2: Describe how switch statements work in JavaScript. When should you use a switch statement instead of if-else?

In JavaScript, a **switch** statement is a control flow structure used to perform different actions based on different conditions. It's often seen as a more organized alternative to a long chain of **if...else if** statements when you are comparing a single variable against multiple specific values.

How a Switch Statement Works

A `switch` statement evaluates an expression once and then compares the result to several "case" labels. If a match is found, the code block associated with that case is executed.

```
switch (expression) {  
  
  case value1:  
  
    // Code to run if expression === value1  
  
    break;  
  
  case value2:  
  
    // Code to run if expression === value2  
  
    break;  
  
  default:  
  
    // Code to run if no cases match  
  
}
```

Key Components

- The Expression: The value you are checking (e.g., a variable like `dayOfWeek`).
- Case: Each `case` defines a possible value for the expression. JavaScript uses strict equality (`===`), meaning the types must match (e.g., the string `"5"` will not match the number `5`).
- Break: This keyword exits the switch block. Without it, the code will "fall through" and execute the next case regardless of whether it matches.
- Default: This is optional but recommended. It runs if none of the cases match the expression, acting like the final `else` in an if-else chain.

Switch vs. If-Else: When to use which?

While both can often achieve the same result, choosing the right one improves code readability and maintainability.

Use `switch` when:

- You have a single variable with many discrete values: If you are checking one variable against 4 or more specific constants (like days of the week, user roles, or command names), a `switch` is much cleaner.
- Readability is a priority: `switch` statements create a tabular, easy-to-read list of outcomes.
- You want "Fall-through" logic: Sometimes you want multiple cases to trigger the same code. You can stack cases to avoid repeating logic.

Use `if-else` when:

- You have complex conditions: If you need to check ranges (e.g., `age > 18 && age < 30`) or use logical operators like `&&` or `||`, `if-else` is necessary.
- You are evaluating different variables: If your first condition checks `isAdmin` and your second checks `loginAttempts`, a `switch` won't work.
- You only have 1 or 2 conditions: A simple `if-else` is less verbose for small logic checks.

Feature	switch Statement	if-else Statement
Best For	Many discrete values of one variable	Complex logic and ranges
Equality Type	Strict Equality (<code>===</code>)	Flexible (can use <code>==</code> , <code>===</code> , <code>></code> , etc.)
Readability	High for long lists of constants	High for branched logic
Performance	Slightly faster for very large lists	Better for general purpose conditions

Lab Assignment

Task1:

Write a JavaScript program to check if a number is positive, negative, or zero using an if-else statement.

```
let num = -5;
```

```
if (num > 0) {
```

```
    // Executes if the condition (num > 0) is true
```

```
    console.log("The number is positive.");
```

```
} else if (num < 0) {
```

```
    // Executes if the first condition is false AND (num < 0) is true
```

```
    console.log("The number is negative.");
```

```
} else {
```

```
    // Executes if both above conditions are false (the number must be 0)
```

```
    console.log("The number is zero.");
```

```
}
```

```
//Logic:
```

if (num > 0): The engine first checks if the number is greater than zero. If it is, it prints the message and skips the rest of the block.

else if (num < 0): If the first check fails, it then checks if the number is less than zero.

else: If neither of the mathematical conditions are met, the only logical remaining possibility is that the number is exactly 0.

Task2:

Create a JavaScript program using a switch statement to display the day of the week based on the user input (e.g., 1 for Monday, 2 for Tuesday, etc.).

A `switch` is the perfect tool for this because we are looking for exact matches for a single variable (`dayNumber`).

// Change this value to test (1-7)

```
let dayNumber = 4;
```

```
let dayName;
```

```
switch (dayNumber) {
```

```
  case 1:
```

```
    dayName = "Monday";
```

```
    Break;
```

```
  case 2:
```

```
    dayName = "Tuesday";
```

```
    Break;
```

```
  case 3:
```

```
    dayName = "Wednesday";
```

```
    Break;
```

```
  case 4:
```

```
    dayName = "Thursday";
```

```
    break;
```

```
  case 5:
```

```
    dayName = "Friday";
```

```
    Break;
```

```
  case 6:
```

```
    dayName = "Saturday";
```

```
    break;

case 7:

    dayName = "Sunday";

    break;

default:

    dayName = "Invalid input! Please enter a number between 1 and 7.";

}

console.log("The day is: " + dayName);
```

Important Concepts in this Program:

- **case**: These act like "checkpoints." If **dayNumber** is **4**, the code jumps straight to **case 4**.
- **break**: This is crucial. It tells JavaScript to stop executing the switch once it finds the match. Without it, the code would "fall through" and execute all subsequent cases (e.g., if you hit case 4, it would also run cases 5, 6, and 7).
- **default**: This handles "bad" data. If the user enters **10** or **"Apple"**, the **default** block ensures the program doesn't just fail silently.
- **Strict Equality**: Remember that **switch** uses **===**. If your input is a string **"1"** but your case is a number **1**, it will not match.

Loops (For, While, Do-While)

Theory Assignment

Question 1: Explain the different types of loops in JavaScript (for, while, do-while). Provide a basic example of each.

Loops are essential in JavaScript for executing a block of code repeatedly. They help automate repetitive tasks and iterate through data collections like arrays.

1. The `for` Loop

The `for` loop is the most commonly used loop. It is ideal when you know exactly how many times you want the code to run. It consists of three parts: initialization, condition, and increment/decrement.

// Syntax: for (initialization; condition; increment)

```
for (let i = 1; i <= 5; i++) {  
  
  console.log("Iteration number: " + i);  
  
}
```

- Initialization (`let i = 1`): Sets the starting point.
- Condition (`i <= 5`): The loop runs as long as this is true.
- Increment (`i++`): Increases the counter after each loop.

2. The `while` Loop

The `while` loop is best used when you don't know exactly how many times the loop should run, but you have a specific condition that must remain true for it to continue.

```
let count = 1;
```

```
while (count <= 3) {  
  
  console.log("Count is: " + count);  
  
  count++; // Important: you must manually update the condition variable
```

```
}
```

Caution: If you forget to update the variable (like `count++`), you will create an infinite loop, which can crash your browser or program.

3. The `do-while` Loop

The `do-while` loop is a variation of the `while` loop. The key difference is that it always executes the code block at least once before checking if the condition is true.

```
let score = 0;
```

```
do {
```

```
  console.log("This message appears at least once.");
```

```
  score++;
```

```
} while (score < 0); // The condition is false, but the code above already ran.
```

Loop Type	When to Use	Key Characteristic
for	You know the iteration count.	Keeps all control logic in one line.
while	You're waiting for a condition to change.	Checks the condition before running.
do-while	You need to run the code at least once.	Check the condition after running.

Question 2: What is the difference between a while loop and a do-while loop?

The fundamental difference between a `while` loop and a `do-while` loop is when the condition is evaluated. Think of it as the "Check-then-Act" approach vs. the "Act-then-Check" approach.

1. The `while` Loop (Entry-Controlled)

In a `while` loop, the condition is tested before the code block executes. If the condition is false right from the start, the code inside the loop will never run.

Example:

```
let fuel = 0;

while (fuel > 0) {

  console.log("Driving...");

  fuel--;

}

// Result: Nothing happens because fuel is not > 0.
```

2. The `do-while` Loop (Exit-Controlled)

In a `do-while` loop, the code block is executed first, and the condition is tested afterwards. This guarantees that the code inside the loop will run at least once, regardless of whether the condition is true or false.

Example:

```
let fuel = 0;

do {

  console.log("Driving at least one block...");

  fuel--;

} while (fuel > 0);

// Result: Prints "Driving at least one block..." once.
```

Feature	while Loop	do-while Loop
Check Timing	Check conditions before execution.	Checks the condition after execution.
Minimum Runs	0 (may never run).	1 (always runs at least once).
Control Type	Entry-controlled.	Exit-controlled.
Common Use	Reading data until the end of a file.	Prompting a user for input until it's valid.

Lab Assignment

Task 1:

Write a JavaScript program using a for loop to print numbers from 1 to 10.

To print numbers from 1 to 10 in JavaScript, the **for** loop is the most efficient choice because we have a clearly defined start point, end point, and increment step.

The Program

// Start at 1; Run while i is 10 or less; Increase i by 1 each time

```
for (let i = 1; i <= 10; i++) {
    console.log(i);
}
```

How it Works Step-by-Step

A **for** loop follows a specific lifecycle to ensure the code runs exactly the number of times you intend.

1. Initialization (**let i = 1**): This happens only once when the loop begins. We create a variable **i** and set it to 1.
2. Condition (**i <= 10**): Before every iteration, JavaScript checks this. If it's **true**, the loop runs. If it's **false**, the loop stops.
3. Execution: The code inside the curly braces **{ ... }** runs, printing the current value of **i** to the console.
4. Increment (**i++**): After the code block runs, **i** is increased by 1. The loop then goes back to Step 2 to check the condition again.

common Variation: Counting Backwards

If you wanted to print from 10 down to 1, you would simply flip the logic:

```
for (let i = 10; i >= 1; i--) {  
  
  console.log(i);  
  
}
```

Task 2:

Create a JavaScript program that uses a while loop to sum all even numbers between 1 and 20.

To sum even numbers between 1 and 20 using a **while** loop, we need a counter to track our progress and an accumulator variable to store the running total.

```
let count = 1;    // Start at 1  
  
let sum = 0;      // Accumulator to store the result  
  
while (count <= 20) {  
  
  // Check if the number is even using the modulo operator (%)  
  
  if (count % 2 === 0) {
```

```
    sum += count; // Add the even number to our sum
  }

  count++; // Increment the counter to avoid an infinite loop
}

console.log("The sum of even numbers between 1 and 20 is: " + sum);
```

This program combines a loop with a conditional statement to filter out odd numbers.

- The Modulo Operator (%): We use `count % 2 === 0` to check if a number is even. This operator finds the remainder of a division. If a number divided by 2 has a remainder of 0, it is even.
- The Accumulator (`sum += count`): This is shorthand for `sum = sum + count`. Every time the loop finds an even number, it "stacks" it onto the previous total.
- The Exit Condition: The loop checks `count <= 20` before every run. Once `count` reaches 21, the condition becomes false and the program moves to the `console.log` line.

Task 3:

Write a do-while loop that continues to ask the user for input until they enter a number greater than 10.

A `do-while` loop is the perfect choice for this task because you want to show the prompt to the user at least once, regardless of what they might have entered previously.

In a real browser environment, we use the `prompt()` function to get user input. Since `prompt()` returns a string, we wrap it in `Number()` to ensure we are comparing actual numbers.

```
let userInput;
```

```
do {
```

```
  // 1. The code inside 'do' runs first
```

```
  userInput = Number(prompt("Enter a number (must be greater than 10 to stop):"));
```

```
// 2. Check if the input is actually a number
```

```
if (isNaN(userInput)) {  
    alert("That's not a valid number! Please try again.");  
}
```

```
} while (userInput <= 10 || isNaN(userInput));
```

```
// 3. The loop repeats if the number is 10 or less, OR if it's not a number
```

```
console.log("Success! You entered: " + userInput);  
alert("Thank you! Loop finished.");
```

Functions

Question 1: What are functions in JavaScript? Explain the syntax for declaring and calling a function.

A function is a reusable block of code designed to perform a particular task. Think of a function like a kitchen appliance: you provide the ingredients (input), it performs a specific process, and it gives you a result (output).

Functions allow you to write code once and use it many times, which makes your programs more organized and easier to maintain.

1. Declaring a Function

Declaring a function is like defining a recipe. You are telling JavaScript what the function is named and what it should do when it is eventually used.

```
function nameOfFunction(parameter1, parameter2) {  
  
    // The code to be executed (function body)  
  
    return parameter1 + parameter2;    }  

```

- **function** keyword: Tells JavaScript you are creating a function.
- Name: The name you'll use to run the function later (e.g., **calculateTotal**).
- Parameters: Variables listed inside parentheses () that act as placeholders for the data you will pass into the function.
- Curly Braces { }: Everything inside these braces is the "body" of the function.
- **return** statement: (Optional) This stops the function and sends a value back to wherever the function was called.

2. Calling (Invoking) a Function

Declaring a function doesn't run the code inside it. To actually execute the code, you must call (or "invoke") the function by using its name followed by parentheses.

The Syntax

```
// Calling the function and passing in "Arguments"
```

```
nameOfFunction(10, 5);
```

Arguments: The actual values you pass into the function (in this case, **10** and **5**). These values fill the spots created by the parameters.

Question 2: What is the difference between a function declaration and a function expression?

In JavaScript, the primary difference between a function declaration and a function expression boils down to hoisting and syntax.

Think of a declaration as "pre-defining" a tool before the program starts, whereas an expression is "creating" a tool on the fly and storing it in a variable.

1. Function Declaration

A declaration is a standalone statement. It starts with the **function** keyword and must have a name.

```
function greet() {  
    return "Hello!"; }  
}
```

Key Feature: Hoisting

Declarations are hoisted, meaning JavaScript moves them to the top of their scope before the code runs. You can call the function before it appears in your code.

```
console.log(sayHi()); // This works!
```

```
function sayHi() {  
    return "Hi!"; }  
}
```

2. Function Expression

An expression occurs when you create a function and assign it to a variable. These functions can be anonymous (no name).

```
const greet = function() {
```

```
return "Hello!";    };
```

Key Feature: Not Hoisted

Function expressions are not hoisted. You cannot use them before they are defined in the code because the variable is treated like any other `let` or `const` variable.

```
console.log(sayHi()); // ReferenceError!
```

```
const sayHi = function() {  
    return "Hi!";    };
```

Feature	Function Declaration	Function Expression
Syntax	function name() { ... }	const name = function() { ... };
Hoisting	Yes (can call before defining).	No (must define before calling).
Structure	Standalone statement.	Part of a variable assignment.
Naming	Must have a name.	Can be anonymous.

Question 3: Discuss the concept of parameters and return values in functions.

In JavaScript functions, parameters and return values represent the input and output of your logic. If a function is a machine, parameters are the raw materials you feed into it, and the return value is the finished product that comes out.

1. Parameters: The Inputs

Parameters are placeholders defined in the function declaration. When you call the function, you provide actual values called arguments to fill those placeholders.

How they work:

- Definition: You can define multiple parameters by separating them with commas.
- Scope: Parameters act as local variables inside the function; they don't exist outside of it.
- Default Parameters: In modern JavaScript, you can set a "fallback" value in case an argument isn't provided.

// 'name' and 'time Of Day' are parameters

```
function greet(name, timeOfDay = "Day") {  
    console.log(`Good ${timeOfDay}, ${name}!`);  
}
```

// "Alice" and "Morning" are arguments

```
greet("Alice", "Morning");    // Output: Good Morning, Alice!
```

```
greet("Bob");                // Output: Good Day, Bob! (uses default)
```

2. Return Values: The Outputs

The **return** statement tells the function to stop executing and "hand back" a value to the code that called it.

Key Rules of Return:

- Immediate Exit: As soon as a **return** is hit, the function stops. Any code written after it inside the function will be ignored.
- Single Value: A function can only return one "thing" (though that thing can be an object or an array containing many values).
- Undefined: If you don't include a **return** statement, the function automatically returns **undefined** by default.

```
function calculateArea(width, height) {  
    let area = width * height;  
    return area; // Sends the result back  
}
```

```
// The value 'returned' is stored in the variable 'myRoom'
```

```
let myRoom = calculateArea(10, 12);
```

```
console.log(myRoom); // 120
```

Concept	Purpose	Analogy
Parameter	Defines what data the function <i>expects</i> .	The slots in a toaster.
Argument	The actual data you <i>provide</i> during the call.	The specific bread you put in.
Return Value	The <i>result</i> sent back after processing.	The toasted bread popping out.

Lab Assignment

Task1:

Write a function `greetUser` that accepts a user's name as a parameter and displays a greeting message (e.g., "Hello, John!").

// Function declaration

```
function greetUser(name) {
```

// We use string concatenation or template literals to include the name

```
  console.log("Hello, " + name + "!");    }
```

// Calling the function with an argument

```
greetUser("John"); // Output: Hello, John!
```

```
greetUser("Sarah"); // Output: Hello, Sarah!
```

Task2:

Create a JavaScript function calculateSum that takes two numbers as parameters, adds them, and returns the result.

```
function calculateSum(num1, num2) {  
    // Add the two parameters and store them in a variable  
    let result = num1 + num2;  
    // Return the value to the caller  
    return result; }  
  
// Calling the function and storing the result in a variable  
let total = calculate Sum(5, 10);  
console.log("The total is: " + total);    // Output: The total is: 15
```

Arrays

Theory Assignment

Question 1: What is an array in JavaScript? How do you declare and initialize an array?

An array is a single variable used to store an ordered list of multiple values. While a regular variable can hold one piece of data (like a single name), an array can hold hundreds or thousands of items (like a list of names) under one name.

Key Characteristics:

- Ordered: Every item in an array has a specific position (index).
- Zero-Indexed: The first item starts at position 0, the second at 1, and so on.
- Flexible: A single array can store different data types (strings, numbers, booleans) at the same time.

1. Declaring and Initializing an Array

There are two primary ways to create an array in JavaScript.

Method A: Array Literal (Recommended)

This is the most common and concise way to create an array using square brackets `[ ]`.

JavaScript

```
// Initializing an array with values
```

```
let fruits = ["Apple", "Banana", "Cherry"];
```

```
// Declaring an empty array to fill later
```

```
let todoList = [];
```

Method B: The `new Array()` Constructor

You can also use the built-in `Array` object, though this is less common in modern coding.

JavaScript

```
let colors = new Array("Red", "Green", "Blue");
```

2. Accessing and Modifying Array Items

You interact with an array using its index inside square brackets.

JavaScript

```
let cars = ["Toyota", "Honda", "Ford"];
```

```
// Accessing the first item (Index 0)
```

```
console.log(cars[0]); // Output: Toyota
```

```
// Changing an item
```

```
cars[1] = "Tesla";
```

```
console.log(cars); // Output: ["Toyota", "Tesla", "Ford"]
```

```
// Checking the length (how many items)
```

```
console.log(cars.length); // Output: 3
```

Question 2: Explain the methods `push()`, `pop()`, `shift()`, and `unshift()` used in arrays.

`push()` — Adds to the end

The `.push()` method adds one or more elements to the very end of an array. It also returns the new length of the array.

```
let snacks = ["Chips", "Cookies"];
```

```
snacks.push("Chocolate");
```

```
console.log(snacks); // ["Chips", "Cookies", "Chocolate"]
```

`pop()` — Removes from the end

The `.pop()` method removes the last element from an array. It returns the item that was removed, allowing you to store it in a variable if needed.

```
let snacks = ["Chips", "Cookies", "Chocolate"];

let lastItem = snacks.pop();

console.log(lastItem); // "Chocolate"

console.log(snacks); // ["Chips", "Cookies"]
```

2. Interacting with the START of the Array

`unshift()` — Adds to the start

The `.unshift()` method adds one or more elements to the beginning of an array. Because it adds to the start, it has to "re-index" all existing items (move them one spot to the right).

```
let queue = ["Bob", "Charlie"];

queue.unshift("Alice");

console.log(queue); // ["Alice", "Bob", "Charlie"]
```

`shift()` — Removes from the start

The `.shift()` method removes the first element from an array. Just like `pop()`, it returns the item it removed.

```
let queue = ["Alice", "Bob", "Charlie"];

let firstInLine = queue.shift();

console.log(firstInLine); // "Alice"

console.log(queue); // ["Bob", "Charlie"]
```

Method	Action	Position	Affects Length?
push()	Add	End	Yes (+1)
pop()	Remove	End	Yes (-1)
unshift()	Add	Start	Yes (+1)
shift()	Remove	Start	Yes (-1)

Lab Assignment

Task1:

- **Declare an array of fruits (["apple", "banana", "cherry"]). Use JavaScript to:**
- **Add a fruit to the end of the array.**
- **Remove the first fruit from the array.**
- **Log the modified array to the console.**

// 1. Declare and initialize the array

```
let fruits = ["apple", "banana", "cherry"];
```

// 2. Add a fruit to the end of the array

```
fruits.push("orange");
```

// 3. Remove the first fruit from the array

```
fruits.shift();
```

// 4. Log the modified array

```
console.log(fruits);
```

Initial State: `["apple", "banana", "cherry"]`

After `.push("orange")`: The new item is added to the very last position.

Result: `["apple", "banana", "cherry", "orange"]`

After `.shift()`: The item at index 0 ("apple") is removed, and all remaining items move "left" to fill the gap.

Result: `["banana", "cherry", "orange"]`

Task2:

- **Write a program to find the sum of all elements in an array of numbers.**

1. Using a `for` Loop (Traditional Approach)

This is the most fundamental way to sum an array. It gives you full control over the iteration process.

```
const numbers = [10, 20, 30, 40, 50];  
  
let sum = 0; // Initialize the accumulator  
  
for (let i = 0; i < numbers.length; i++) {  
    sum += numbers[i]; // Add the current element to the sum  
}  
  
console.log("The sum is:", sum); // Output: 150
```

2. Using the `reduce()` Method (Modern Approach)

In modern JavaScript, the `.reduce()` method is the most elegant way to boil an entire array down to a single value (like a sum).

```
const numbers = [10, 20, 30, 40, 50];
```

// The reduce method takes a "callback" function with an accumulator and the current value

```
const totalSum = numbers.reduce((accumulator, currentValue) => {  
  return accumulator + currentValue;  
}, 0);    // 0 is the starting value for the accumulator  
  
console.log("The sum using reduce is:", totalSum);
```

3. Using a `for...of` Loop (Readable Approach)

If you don't need the index numbers and just want the values, the `for...of` loop is very clean and easy to read.

```
const numbers = [10, 20, 30, 40, 50];  
  
let sum = 0;  
  
for (const num of numbers) {  
  sum += num;    }  
  
console.log("The sum using for...of is:", sum);
```

Objects

Theory Assignment

Question 1: What is an object in JavaScript? How are objects different from arrays?

An object is a standalone entity that stores data in key-value pairs. If an array is a numbered list, an object is like a labeled filing cabinet or a dictionary. Instead of using a number (index) to find a piece of information, you use a descriptive name (key).

1. Object Syntax

Objects are defined using curly braces `{ }`. Each "key" (also called a property) is linked to a "value."

```
const car = {  
  brand: "Tesla",  
  model: "Model 3",  
  year: 2023,  
  is Electric: true    };
```

// Accessing data using "Dot Notation"

```
console.log(car.brand); // Output: Tesla
```

// Accessing data using "Bracket Notation"

```
console.log(car["model"]); // Output: Model 3
```

Objects vs. Arrays: The Core Differences

While both are used to store collections of data, they serve very different purposes based on how that data is structured.

Feature	Array []	Object {}

Data Format	Ordered list of values.	Unordered set of key-value pairs.
Access Method	Accessed by numerical index (0, 1, 2...).	Accessed by a key name (string).
Best For	Lists of similar items (e.g., a list of names).	Detailed descriptions of one thing (e.g., a user profile).
Ordering	Elements stay in a specific order.	Properties do not have a guaranteed order.

Question 2: Explain how to access and update object properties using dot notation and bracket notation.

In JavaScript, you can interact with object properties in two primary ways: Dot Notation and Bracket Notation. Choosing between them depends on whether you know the property name ahead of time or if it needs to be determined dynamically.

1. Dot Notation

Dot notation is the most common and readable way to access or update properties. You use a period (.) followed by the property name.

How to use it:

- Accessing: `object.property`
- Updating: `object.property = newValue`

```
const user = {
  name: "Sarah",
```

```
age: 28    };
```

// Accessing

```
console.log(user.name); // "Sarah"
```

// Updating

```
user.age = 29;
```

```
console.log(user.age); // 29
```

2. Bracket Notation

Bracket notation uses square brackets `[ ]` and the property name as a string. This is more powerful and flexible than dot notation.

How to use it:

- Accessing: `object["property"]`
- Updating: `object["property"] = newValue`

```
const user = {
```

```
  "home address": "123 Maple St",
```

```
  age: 28    };
```

// Accessing a property with a space

```
console.log(user["home address"]);
```

// Updating

```
user["age"] = 30;
```

3. When to use Bracket Notation?

While dot notation is cleaner, bracket notation is mandatory in two specific scenarios:

A. When the property name is dynamic (stored in a variable)

If you don't know the key name until the code is running, you must use brackets.

```
const settings = {
  theme: "dark",
  notifications: true };

let key = "theme";

console.log(settings.key); // undefined (looks for a property actually named "key")
console.log(settings[key]); // "dark" (looks for the value stored in the variable 'key')
```

B. When the key has special characters or spaces

Keys like `"user-id"` or `"first name"` cannot be accessed with a dot.

```
const data = { "user-id": 505 };
```

```
// console.log(data.user-id); // Error! (JavaScript thinks you are subtracting 'id' from 'data.user')
```

```
console.log(data["user-id"]); // Works perfectly
```

Feature	Dot Notation	Bracket Notation
Syntax	obj.key	obj["key"]
Readability	High	Medium
Use Variables?	No	Yes
Special Characters?	No	Yes

Lab Assignment

Task:

Create a JavaScript object car with properties brand,model,and year.

UseJavaScript to:

- Access and print the car's brand and model.
- Update the year property.
- Add a new property color to the car object.

// 1. Create the car object

```
const car = {  
  brand: "Toyota",  
  model: "Camry",  
  year: 2020  };
```

// 2. Access and print the brand and model

```
console.log("Brand:", car.brand);  
console.log("Model:", car.model);
```

// 3. Update the year property

```
car.year = 2024;
```

// 4. Add a new property 'color'

```
car.color = "Midnight Blue";
```

// Final log to see the updated object

```
console.log(car);
```

JavaScript Events

Theory Assignment

Question 1: What are JavaScript events? Explain the role of event listeners.

1. What are JavaScript Events?

An event is an action or occurrence that happens in the system you are programming. The browser generates a signal when an event occurs, and provides a mechanism so some action can be taken automatically (such as running a piece of code).

Common examples of events include:

- Mouse Events: Clicking a button (`click`), hovering over an element (`mouseover`), or dragging an item.
- Keyboard Events: Pressing a key (`keydown`) or releasing a key (`keyup`).
- Form Events: Submitting a form (`submit`) or changing the value of an input (`change`).
- Window Events: Resizing the browser window (`resize`) or the page finishing its load process (`load`).

2. The Role of Event Listeners

An Event Listener is a procedure in JavaScript that waits for an event to occur. You can think of it as a "guard" stationed at a specific part of your HTML, waiting for a specific trigger to happen.

When the trigger occurs, the listener "hears" it and executes a specific function (often called an Event Handler).

The Three Parts of an Event Listener

To set up a listener, you generally need three pieces of information:

1. The Target: Which HTML element are we watching? (e.g., a specific button).
2. The Type: What specific action are we waiting for? (e.g., a "click").
3. The Function: What should happen when the event is detected? (e.g., show a popup).

3. How it looks in Code

The modern way to handle events is using the `addEventListener()` method.

JavaScript

```
const btn = document.querySelector('#submit-btn');  
  
// Target // Listener // Type // Function (Handler)  
  
btn. addEventListener('click', function() {
```

```
    console.log("The button was clicked!");    });
```

4. Why Use Event Listeners?

- Separation of Concerns: You keep your HTML (structure) and JavaScript (logic) in separate files, making the code easier to read and maintain.
- Multiple Listeners: You can attach multiple listeners to the same element. For example, one click could save data AND change the button's color.
- Dynamic Control: You can add or remove listeners while the program is running (using `removeEventListener`). This is useful for "disabling" buttons after they have been clicked once.

Question 2: How does the `addEventListener()` method work in JavaScript? Provide an example.

The `addEventListener()` method is the standard way to handle interactions on a webpage. It instructs the browser to "listen" for a specific event on a specific HTML element and execute a function whenever that event occurs.

1. The Anatomy of `addEventListener`

The method takes three main arguments (though the third is optional):

```
element.addEventListener(event,function,useCapture);
```

1. Event: A string representing the event name (e.g., `"click"`, `"submit"`, `"keydown"`).

2. Function: The code that runs when the event is triggered (the "callback" or "handler").
3. UseCapture (Optional): A boolean that determines if the event should be captured during the "capture" phase or "bubble" phase.

2. Practical Example

Imagine you have a simple light bulb setup where clicking a button toggles the background color of a `div`.

HTML

```
<div id="box" style="width: 100px; height: 100px; background: gray;"></div>  
<button id="toggleBtn">Change Color</button>
```

JavaScript

// 1. Select the elements

```
const box = document.getElementById('box');  
const btn = document.getElementById('toggleBtn');
```

// 2. Define the action (Handler)

```
function turnRed() {  
  box.style.backgroundColor = 'red';  
  console.log("The box is now red!"); }  

```

// 3. Attach the listener

```
btn.addEventListener('click', turnRed);
```

3. How the Browser Processes It

When you call `addEventListener`, the following happens behind the scenes:

- Registration: The browser adds the function to a hidden list of "observers" attached to that specific DOM element.

- Idle Waiting: The JavaScript engine doesn't "freeze" or wait there; it continues running other code.
- Event Trigger: When the user clicks, the browser's event system creates an Event Object and places it in the Task Queue.
- Execution: The Event Loop picks up the task and pushes the `turnRed` function onto the call stack to be executed.

4. Key Advantages

- Multiple Listeners: You can attach many different functions to the same event on the same element without them overwriting each other.
- Removal: You can stop listening at any time using `removeEventListener(event, function)`. This is vital for performance in complex applications.
- Clean HTML: You don't have to put messy `onclick=""` attributes inside your HTML tags.

Lab Assignment

Task

Create a simple webpage with a button that, when clicked, displays an alert saying "Button clicked!" using JavaScript event listeners.

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <title>Button Click Example</title>

</head>

<body>

  <button id="myButton">Click Me!</button>
```

```
<script>
```

```
    // Find the button in the document using its ID
```

```
    const btn = document.getElementById('myButton');
```

```
    btn.addEventListener('click', function() {
```

```
        // Action to perform when clicked
```

```
        alert('Button clicked!');    });
```

```
</script>
```

```
</body>
```

```
</html>
```

DOM Manipulation

Theory Assignment

Question 1: What is the DOM (Document Object Model) in JavaScript? How does JavaScript interact with the DOM?

The Document Object Model (DOM) is a programming interface for web documents. It represents the structure of an HTML document as a tree of objects, where every single part of the page—from the `<body>` tag down to a piece of text or a button—is a "node" that JavaScript can manipulate.

When a browser loads a webpage, it takes the raw HTML code and transforms it into this logical tree structure.

1. The Structure of the DOM

Think of the DOM as a family tree. The `window` is the grandparent, the `document` is the parent, and HTML tags are the children.

- Document: The root of the tree.
- Elements: Tags like `<div>`, `<h1>`, or `<p>`.
- Attributes: Information inside tags, like `href` or `src`.
- Text: The actual content inside the elements.

2. How JavaScript Interacts with the DOM

JavaScript is the "hand" that moves the pieces within the DOM tree. It interacts with the DOM through four main actions:

A. Selecting Elements (Finding)

Before you can change something, you have to find it. JavaScript provides several methods to "grab" elements.

- `document.getElementById('id-name')`
- `document.querySelector('.class-name')`

B. Changing Content and Styles (Modifying)

Once you have selected an element, you can change how it looks or what it says.

```
const title = document.querySelector('h1');  
  
title.textContent = "Welcome to my Site!"; // Changes the text  
  
title.style.color = "blue";           // Changes the CSS
```

C. Adding or Removing Elements (Creating)

JavaScript can build new parts of the page on the fly.

```
const newPost = document.createElement('p');  
  
newPost.textContent = "This is a new paragraph.";  
  
document.body.appendChild(newPost); // Adds it to the bottom of the page
```

D. Listening for Events (Reacting)

As we discussed earlier, the DOM allows JavaScript to wait for user interactions (clicks, scrolls, typing) and respond accordingly.

3. Why the DOM is Important

Without the DOM, JavaScript would have no way to "see" or "talk" to your HTML. It would just be a logic engine with no screen to display its results. The DOM is what makes the web dynamic—it allows a page to update without requiring a full refresh.

Question 2: Explain the methods `getElementById()`, `getElementsByClassName()`, and `querySelector()` used to select elements from the DOM.

1. `getElementById()`

This is the most direct and efficient way to select an element. It looks for a specific HTML element that has a unique `id` attribute.

- Returns: A single element (object).
- Behavior: Since IDs must be unique on a page, it only ever returns the first match it finds. If no match is found, it returns `null`.

JavaScript

```
// HTML: <h1 id="main-title">Hello</h1>
```

```
const title = document.getElementById("main-title");
```

```
title.style.color = "blue";
```

2. getElementsByClassName()

This method searches for all elements that have a specific `class` attribute.

- Returns: An HTMLCollection (an array-like list of elements).
- Behavior: Because multiple elements can share the same class, this returns a collection, even if only one element is found. You must use an index (like `[0]`) to access a specific element.

JavaScript

```
// HTML: <li class="item">List 1</li> <li class="item">List 2</li>
```

```
const items = document.getElementsByClassName("item");
```

```
// To change the first one:
```

```
items[0].textContent = "Updated Item";
```

```
// To change all of them, you must loop:
```

```
for (let i = 0; i < items.length; i++) {  
    items[i].style.fontWeight = "bold";  
}
```

3. querySelector()

This is the most modern and versatile method. It uses CSS Selector syntax to find elements, allowing you to search by ID, class, tag name, or even complex relationships (like "the first child of a div").

- Returns: The first element that matches the selector.
- Behavior: If you want to select *all* matching elements, you would use `querySelectorAll()` instead.

JavaScript

// Selecting by ID (use #)

```
const header = document.querySelector("#main-header");
```

// Selecting by Class (use .)

```
const firstBtn = document.querySelector(".btn-primary");
```

// Selecting by Tag and Attribute

```
const emailInput = document.querySelector("input[type='email']");
```

Method	Selects By	Returns	Flexibility
getElementById	ID only	Single Element	Low (Fastest)
getElementsByClassName	Class only	HTMLCollection	Medium
querySelector	Any CSS Selector	Single Element	High (Most common)

Lab Assignment

Task:

Create an HTML page with a paragraph (<p>) that displays "Hello, World!".

Use JavaScript to:

Change the text inside the paragraph to "JavaScript is fun!".

Change the color of the paragraph to blue.

```
<!DOCTYPE html>
```

```
<html lang="en">

<head>

  <meta charset="UTF-8">

  <title>DOM Manipulation Exercise</title>

</head>

<body>

  <p id="message">Hello, World!</p>

  <script>

    // 1. Select the paragraph using its ID

    const paragraph = document.getElementById('message');

    // 2. Change the text content

    paragraph.textContent = "JavaScript is fun!";

    // 3. Change the text color using the style property

    paragraph.style.color = "blue";

  </script>

</body>

</html>
```

JavaScript Timing Events (setTimeout, setInterval)

Theory Assignment

Question 1: Explain the setTimeout() and setInterval() functions in JavaScript. How are they used for timing events?

1. setTimeout(): The One-Time Delay

The `setTimeout()` function executes a block of code once after a specified delay.

Syntax:

```
setTimeout(function,delay,param1,param2,...);
```

- Function: The code you want to run.
- Delay: Time in milliseconds (e.g., 1000=1 second).
- Params: Optional arguments to pass to the function.

Example:

JavaScript:

```
console.log("Start");

setTimeout(() => {

    console.log("This appears after 3 seconds");

}, 3000);

console.log("End");
```

2. setInterval(): The Repeated Loop

The `setInterval()` function executes a block of code repeatedly at every specified time interval.

Syntax:

```
setInterval(function,interval);
```

Example:

JavaScript:

```
let count = 0;
```

```
const timer = setInterval(() => {  
    count++;  
    console.log("Seconds passed: " + count);  
}, 1000);
```

3. Stopping the Timers

To stop a timer, you must use `clearTimeout()` or `clearInterval()`. When you create a timer, it returns a unique ID (a number) that you use as a reference to cancel it.

JavaScript:

```
const myTimer = setInterval(() => {  
    console.log("Ticking...");  
}, 1000);
```

// Stop the timer after 5 seconds

```
setTimeout(() => {  
    clearInterval(myTimer);  
    console.log("Timer stopped.");  
}, 5000);
```

Feature	setTimeout()	setInterval()
Repetition	Runs once.	Runs repeatedly.

Purpose	Delays an action.	Automates a recurring task (like a clock).
Termination	Usually stops itself after running.	Must be cleared manually using the ID.

Question 2: Provide an example of how to use `setTimeout()` to delay an action by 2 seconds.

To delay an action in JavaScript, you use the `setTimeout()` function. It takes two main arguments: the function you want to run and the delay in milliseconds.

Since 1000 milliseconds=1 second, you will use 2000 for a two-second delay.

The Basic Example:

```
console.log("Timer started...");

setTimeout(function() {
    console.log("2 seconds have passed!");
}, 2000);

console.log("This message appears immediately, even though it is after the timeout code.");
```

Lab Assignment

Task 1:

Write a program that changes the background color of a webpage after 5 seconds using `setTimeout()`.

```
<!DOCTYPE html>
```

```
<html lang="en">

<head>

  <meta charset="UTF-8">

  <title>Timed Background Change</title>

  <style>

    body {

      transition: background-color 1s; /* Makes the color change smooth */

      font-family: sans-serif;

      display: flex;

      justify-content: center;

      align-items: center;

      height: 100vh;

      margin: 0;

    }

  </style>

</head>

<body>

  <h1>Wait 5 seconds...</h1>

  <script>

    // setTimeout takes two arguments:

    // 1. A function to run

    // 2. The delay in milliseconds (5000ms = 5 seconds)
```

```
        setTimeout(function() {  
            // Targets the body of the document and changes its style  
            document.body.style.backgroundColor = "lightblue";  
            // Updates the text to show the task is done  
            document.querySelector('h1').textContent = "Color Changed!";  
        }, 5000);  
    </script>  
</body>  
</html>
```

Task 2:

Create a digital clock that updates every second using setInterval().

```
<!DOCTYPE html>  
  
<html lang="en">  
  
<head>  
    <meta charset="UTF-8">  
    <title>Simple Digital Clock</title>  
    <style>  
        body {  
            display: flex;  
            justify-content: center;  
            align-items: center;  
            height: 100vh;
```

```
background-color: #2c3e50;

color: #ecf0f1;

font-family: 'Courier New', Courier, monospace;

}
```

```
#clock {

    font-size: 5rem;

    border: 5px solid #bdc3c7;

    padding: 20px;

    border-radius: 10px;

    background-color: #34495e;

}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div id="clock">00:00:00</div>
```

```
<script>
```

```
function updateClock() {

    // 1. Get the current date and time

    const now = new Date();

    // 2. Extract hours, minutes, and seconds

    let hours = now.getHours();

    let minutes = now.getMinutes();

    let seconds = now.getSeconds();
```

```
// 3. Add a leading zero if the number is less than 10

hours = hours < 10 ? '0' + hours : hours;

minutes = minutes < 10 ? '0' + minutes : minutes;

seconds = seconds < 10 ? '0' + seconds : seconds;

// 4. Format the string

const timeString = `${hours}:${minutes}:${seconds}`;

// 5. Update the DOM

document.getElementById('clock').textContent = timeString;

}

// Run the function every 1000ms (1 second)

setInterval(updateClock, 1000);

// Call it once immediately so it doesn't wait 1 second to start

updateClock();

</script>

</body>

</html>
```

JavaScript Error Handling

Theory Assignment

Question 1: What is error handling in JavaScript? Explain the try, catch, and finally blocks with an example.

Error handling is the process of anticipating, detecting, and responding to "exceptions"—errors that occur during the execution of your code. Without error handling, a single mistake (like a network failure or a typo) can cause your entire application to crash and stop running.

In JavaScript, the primary mechanism for this is the `try...catch...finally` statement.

1. The Core Blocks

`try` block

This is where you place the code you want to test. It's the "risky" area where you think an error might occur (e.g., fetching data, parsing JSON, or performing complex math).

`catch` block

If an exception occurs in the `try` block, JavaScript immediately jumps to the `catch` block. This block "catches" the error object, allowing you to handle it gracefully (like showing an alert) instead of letting the script die.

`finally` block

This block is optional. It runs no matter what, regardless of whether an error was caught or not. It is typically used for "cleanup" actions, such as closing a file or stopping a loading spinner.

2. Practical Example: Parsing JSON

Imagine you are receiving data from a server. If the data is "broken" (not valid JSON), the `JSON.parse()` method will throw an error.

JavaScript

```

function processData(jsonString) {
    try {
        console.log("Attempting to parse data...");
        // This will fail if jsonString is not valid JSON
        const data = JSON.parse(jsonString);
        console.log("Success! Data received:", data);
    }
    catch (error) {
        // This runs only if an error occurs above
        console.error("Oops! We hit a snag.");
        console.log("Error Message:", error.message);
    }
    finally {
        // This runs every time
        console.log("Cleanup: Closing connection to server.");
    }
}

// Case 1: Valid JSON
processData('{ "name": "Alice" }');
console.log("---");

// Case 2: Invalid JSON (Missing quotes)
processData('{name: Alice}');

```

3. The `throw` Keyword

Sometimes, a piece of code is mathematically "legal" but logically wrong for your app. You can use the `throw` keyword to manually create an error and trigger the `catch` block.

```
function checkAge(age) {  
  try {  
    if (age < 18) {  
      throw new Error("You must be at least 18 years old.");  
    } console.log("Access granted.");  
  } catch (e) {  
    console.log("Validation Error:", e.message);  
  }  
}  
  
checkAge(15);
```

Block	Required?	When does it run?
try	Yes	Always (the first block to run).
catch	No*	Only if an error occurs in the <code>try</code> block.
finally	No*	Always (runs after <code>try</code> or <code>catch</code> completes).

Question 2: Why is error handling important in JavaScript applications?

Error handling is critical because JavaScript is a single-threaded and event-driven language. In many environments, if a script encounters an unhandled error, the entire execution thread can "crash," leading to a completely frozen or broken user interface.

Here is why error handling is indispensable in modern development:

1. Preventing "Silent Failures" and Crashes

Without error handling, a single typo or a failed network request can stop your script in its tracks. In a web browser, this might mean a button stops working; in a Node.js server, it could mean the entire server shuts down.

2. Improving User Experience (UX)

Users are much more forgiving of an error message than a broken interface. Instead of a blank screen or a button that does nothing when clicked, error handling allows you to show a friendly notification, such as:

- *"Check your internet connection and try again."*
- *"That username is already taken."*

3. Debugging and Maintainability

Error handling allows developers to log exactly what went wrong, where it happened, and why. By using the `error` object inside a `catch` block, you can send technical details to a logging service (like Sentry or LogRocket) to fix bugs before more users encounter them.

JavaScript:

```
try {  
    // Complex logic  
} catch (error) {  
    logErrorToService(error); // Helps developers find the needle in the haystack  
}
```

4. Managing External Dependencies

Modern apps rely on APIs, databases, and third-party libraries. You cannot control if a server goes down or if a database is slow. Error handling acts as a "safety net" for these external uncertainties.

Lab Assignment

Task:

Write a JavaScript program that attempts to divide a number by zero. Use try-catch to handle the error and display an appropriate error message.

```
function divideNumbers(a, b) {  
  try {  
    // In JS, 10 / 0 equals Infinity.  
  
    // We will manually "throw" an error if b is 0 to trigger the catch block.  
  
    if (b === 0) {  
      throw new Error("Division by zero is not allowed!");  
    }  
  
    let result = a / b;  
  
    console.log("Result:", result);  
  } catch (error) {  
    // This block runs if an error occurs in the try block  
  
    console.error("Caught an error:", error.message);  
  } finally {  
    // This block runs no matter what happens
```

```
        console.log("Division attempt completed.");
    }
}

// Test cases

divideNumbers(10, 2); // Successful division

divideNumbers(10, 0); // Triggers the catch block
```

